github.com/mg-i-roc-maz/corsi · microservizi/lezione-7

LEZIONE 7 DI 8 · 3 ORE

Architetture event-driven

orders non chiama più nessuno per l'email: racconta che l'ordine esiste. Chi è interessato ascolta, quando può.

Le tre ore di oggi

Teoria · 1h15

Laboratorio · 1h35

Teoria

Comandi ed eventi, publish/subscribe, RabbitMQ, conferme e at-least-once, consistenza eventuale, cenni a Kafka.

Laboratorio

RabbitMQ nel Compose, orders pubblica OrderCreated, notifications lo consuma. Code e messaggi nella Management UI.

Ultimi 10'

Una domanda per chiudere: «cosa si rompe se…?»

DA DOVE RIPARTIAMO

«Cucitura 3»: dalla lezione 3, l'email di conferma è una riga di log dentro orders.

```
// services/orders/src/routes.ts, lezione 6
const order = store.create(customerEmail, lines);
request.log.info(`Email di conferma a ${customerEmail} ...`);
return reply.code(201).send(order);
```

E se il server di posta è lento? E se domani serve anche un SMS, e un aggiornamento del magazzino?

COMANDI ED EVENTI

Un evento è un fatto: è già successo.

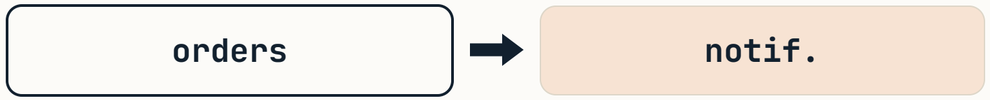
OrderCreated non chiede niente a nessuno. Dice: «l'ordine o-1 esiste». Chi vuole saperlo, ascolta.

Comando o evento?

	Comando	**Evento**
Esempio	**SendEmail**, ReserveStock	**OrderCreated**, StockReserved
Tempo verbale	imperativo: «fai»	passato: «è successo»
Destinatari	uno, scelto da chi manda	zero, uno, molti: chi si iscrive
Può essere rifiutato?	sì: il destinatario decide	no: è un fatto
Chi conosce chi	chi manda conosce chi riceve	chi pubblica non conosce nessuno

Sincrono o asincrono?

HTTP: orders aspetta



- notifications lento: l'ordine è lento
- notifications giù: e l'ordine?
- un nuovo destinatario: cambia orders

Evento: orders prosegue



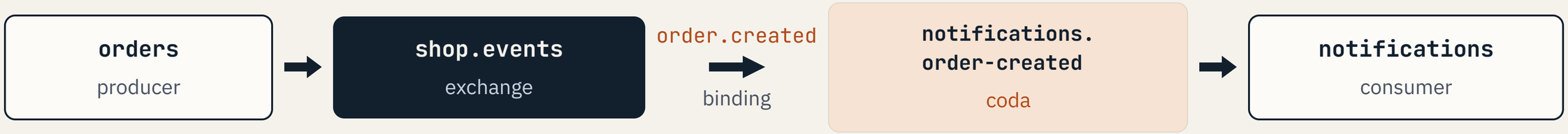
- orders risponde in pochi ms
- notifications giù: il messaggio aspetta
- un nuovo destinatario: una coda in più

Tre modi di dipendere da un altro servizio

Accoppiamento	Domanda	Con un evento
Nel tempo	Deve essere acceso adesso?	No: la coda aspetta
Nello spazio	Devo sapere dov'è e chi è?	No: conosco solo l'exchange
Nel formato	Devo capire i suoi dati?	Sì: il contratto resta

Gli eventi tolgono le prime due dipendenze, **non la terza**: il contenuto di OrderCreated è un contratto come un'API.

Un broker in mezzo



Pezzo	Cosa fa
exchange	riceve i messaggi e li **smista**: non li conserva
binding	la regola: quali messaggi vanno in quale coda (routing key)
coda	**conserva** i messaggi finché qualcuno non li conferma

Tipi di exchange

Tipo	Smista per	Esempio
direct	routing key uguale al binding	«email» → coda email
topic	routing key con caratteri jolly	**order.*** → tutti gli eventi degli ordini
fanout	niente: tutte le code collegate	un avviso a tutti

`order.*`

una parola: **order.created**, order.shipped

`#`

zero o più parole: **tutto**

`*.changed`

product.changed, price.changed

Pubblicare: orders

```
const ch = await model.createConfirmChannel();
await ch.assertExchange("shop.events", "topic",
                        { durable: true });

ch.publish("shop.events", "order.created",
  Buffer.from(JSON.stringify(event)), {
    persistent: true,
    mandatory: true,
    messageId: event.id,
    correlationId,
  }, (err) ⇒ ...); // conferma del broker
```

Opzione	Perché
ConfirmChannel	sapere che RabbitMQ l'ha preso
persistent	scritto su disco
mandatory	torna indietro se nessuna coda lo vuole
messageId	riconoscere i doppioni

Consumare: notifications

```
await ch.assertQueue("notifications.order-created",
                     { durable: true });
await ch.bindQueue("notifications.order-created",
                   "shop.events", "order.created");
await ch.prefetch(10);

await ch.consume(queue, async (msg) ⇒ {
  await sendEmail(JSON.parse(msg.content));
  ch.ack(msg); // fatto: toglilo dalla coda
}, { noAck: false });
```

La coda è di chi consuma: la dichiara **notifications**, non orders.

noAck: false: la conferma la diamo noi, a lavoro finito.

Due code o due consumer?

Due code: publish/subscribe

```
coda notifications → email
```

```
coda vendite → statistiche
```

Ognuno riceve la sua copia di ogni evento.

Una coda, due copie: competing consumers

```
notifications #1 → o-1, o-3, o-5
```

```
notifications #2 → o-2, o-4, o-6
```

Il lavoro si **divide**: ogni messaggio a una sola copia.

prefetch: quanti messaggi in mano

prefetch	Effetto
1	un messaggio alla volta: lavoro diviso alla perfezione, più lento
10	fino a 10 non confermati: più veloce, divisione meno equa
nessun limite	una copia si prende tutta la coda, le altre restano a guardare

I messaggi «in mano» a un consumer sono **Unacked** nella UI: consegnati, non ancora confermati.

Se il consumer cade, tornano Ready.

Cosa sopravvive a un riavvio di RabbitMQ?

Dichiarato	Dopo il riavvio
exchange e coda **durable**	ci sono ancora
messaggio **persistent** in una coda durable	**c'è ancora**
messaggio non persistent	**perso**
coda non durable	**persa**, con i suoi messaggi
volume del container cancellato (down -v)	**tutto perso**

Nel Compose: volume **rabbitmq-data** e **hostname: rabbitmq** fisso.

Dire a RabbitMQ com'è andata

Risposta	Quando	Cosa fa RabbitMQ
ack	email mandata	toglie il messaggio dalla coda
nack, requeue	il server di posta è giù	lo rimette in coda: riprova
nack, scarta	il messaggio è illeggibile	lo toglie (lezione 8: DLQ)
nessuna risposta	notifications cade a metà	alla disconnessione lo riconsegna

⚠ Un messaggio che fallisce **sempre**, con requeue, torna in coda per sempre: un ciclo infinito. La soluzione, tentativi contati e Dead Letter Queue, è nella lezione 8.

Quante volte arriva un messaggio?

At-most-once

Al massimo una volta: forse mai.

Conferma prima del lavoro, o Redis Pub/Sub (lezione 6).

At-least-once

Almeno una volta: a volte due.

Conferma dopo il lavoro. È quello che facciamo oggi.

Exactly-once

Esattamente una volta: in rete non esiste.

Si ottiene l'**effetto** con at-least-once + idempotenza.

Il doppione nasce qui: email mandata, notifications cade **prima dell'ack**. RabbitMQ non lo sa: riconsegna.

Il consumer idempotente, in anticipo

```
if (await seen.has(event.id)) {
  ch.ack(msg); // già fatto: conferma e basta
  return;
}
await sendEmail(event);
await seen.add(event.id);
ch.ack(msg);
```

Per questo ogni evento ha un **id** unico, messo da chi pubblica.

Non tutti i consumer ne hanno bisogno: buttare una copia in cache due volte è innocuo. Mandare due email, o scalare due volte le scorte, no.

E dal lato di chi pubblica?

Conferma del broker

Con un ConfirmChannel la callback dice: **RabbitMQ l'ha preso**. Senza, publish() non sa niente.

mandatory

Nessuna coda collegata? Il messaggio **torna indietro**, invece di sparire in silenzio.

⚠ Anche con tutte e due: l'ordine è salvato, poi la pubblicazione fallisce. L'ordine c'è, l'evento no, l'email non partirà mai. È la **doppia scrittura**: lezione 8, Outbox.

Consistenza eventuale

`0 ms`	POST /orders → **201**: l'ordine esiste
`15 ms`	OrderCreated nella coda: l'email **non c'è ancora**
`… 2 min`	notifications è fermo: il messaggio aspetta
`2 min 3 s`	notifications riparte: l'email parte

Il sistema è coerente **prima o poi**, non subito. La domanda: quanto «poi» è accettabile?

Progettare per il «prima o poi»

Dove	Cosa cambia
Interfaccia	«Riceverai un'email di conferma», non «ti abbiamo mandato un'email»
Stato	un ordine può essere «confermato» prima che l'email sia partita
Monitoraggio	la lunghezza delle code è una metrica: se cresce, qualcuno non ce la fa
Ordine dei messaggi	garantito in una coda con un consumer; non con due copie o i nuovi tentativi

E Kafka?

	RabbitMQ	Apache Kafka
Modello	code: il messaggio sparisce dopo l'ack	log: i messaggi restano, per giorni
Rileggere	no	sì: ogni consumer ha il suo segno
Ordine	per coda	per partizione (per chiave)
Instradamento	ricco: exchange, binding, jolly	semplice: topic e partizioni
Pensato per	lavori e notifiche tra servizi	flussi enormi, analisi, ricostruire stati

La busta dell'evento: un contratto

```
{
  "id": "4f1c…", // unico: doppioni
  "type": "OrderCreated",
  "version": 1, // cambia se cambia data
  "occurredAt": "2026-10-27T10:15:02Z",
  "correlationId": "5cbb…", // lezione 4
  "data": { "orderId": "o-1",
            "customerEmail": "anna@example.com",
            "totalCents": 16970, "lines": [ … ] }
}
```

Si può **aggiungere** un campo: chi non lo conosce lo ignora.

Si cambia o si toglie: **nuova versione**, e chi legge la impara prima.

Verifichiamo: eventi e RabbitMQ

1. ReserveStock è un comando o un evento? E StockReserved?
2. Chi dichiara la coda di notifications, e perché proprio lui?
3. Cosa succede a un messaggio pubblicato quando nessuna coda è collegata?
4. Due copie di notifications sulla stessa coda: ricevono tutte e due ogni messaggio?

Verifichiamo: consegna e consistenza

1. Perché l'ack si manda DOPO aver mandato l'email?
2. Come nasce un'email doppia con at-least-once?
3. RabbitMQ è giù quando si crea un ordine: cosa succede oggi?
4. Cosa vuol dire consistenza eventuale, nel nostro negozio?

Glossario

Termine	Significato
Evento, comando	Un fatto già successo; una richiesta di fare qualcosa.
Broker	Il servizio che riceve, smista e conserva i messaggi: RabbitMQ.
Exchange, binding, coda	Chi smista, la regola, chi conserva.
Routing key	L'etichetta del messaggio: order.created.
ack, nack	«Fatto», «non fatto»: tolto o rimesso in coda.
At-least-once	Almeno una volta: niente perso, doppioni possibili.
Competing consumers	Più copie sulla stessa coda si dividono i messaggi.
Consistenza eventuale	Il sistema diventa coerente dopo un po', non subito.

Laboratorio

10'

1 · RabbitMQ

TODO 1: nel Compose.

25'

2 · Pubblicare

TODO 2 e 3: orders e OrderCreated.

25'

3 · Consumare

TODO 4 e 5: notifications.

25'

4 · Esperimenti

Code che aspettano, errori, due copie, cadute.

```
cd microservizi/lezione-7/start → docker compose up --build -d
```

Il repo della lezione

`github.com/mg-i-roc-maz/corsi`

```
cd corsi
git pull
cd microservizi/lezione-6/start
docker compose down
cd ../../lezione-7/start
docker pull rabbitmq:4.1-management
docker compose up --build -d
```

microservizi/lezione-7/

Cartella	Contenuto
`slides.pdf`	Queste slide, in PDF
`start/`	**Si lavora qui**
`solution/`	La soluzione, per riallinearsi
`extra/`	Esercizi extra, test e soluzioni

Nuova dipendenza: **amqplib**. **npm install** in orders e in notifications.

Cosa c'è in start

```
compose.yaml                    TODO 1: rabbitmq
services/orders/src/
  events.ts                     TODO 2: pubblicare
  routes.ts                     TODO 3: dopo l'ordine
services/notifications/         nuovo servizio
  src/broker.ts                 TODO 4: coda e consumo
  src/consume.ts                TODO 5: ack e nack
  src/handlers.ts               OrderCreated → email
  src/mailer.ts                 email finte, nel log
```

notifications è un servizio Fastify come gli altri, ma il suo lavoro arriva dalla **coda**, non da HTTP.

Le rotte servono solo a guardarlo: /notifications, /stats, /health.

1 · TODO 1: RabbitMQ nel Compose

```
rabbitmq:
  image: rabbitmq:4.1-management
  hostname: rabbitmq
  ports:
    - "15672:15672" # Management UI
    - "5672:5672"
  volumes:
    - rabbitmq-data:/var/lib/rabbitmq
  healthcheck:
    test: ["CMD", "rabbitmq-diagnostics", "-q", "ping"]
  restart: unless-stopped
```

Poi togli il commento ai **depends_on** di orders e notifications.

Management UI

http://localhost:15672
guest / guest

Exchanges, Queues: cosa c'è già? Chi l'ha creato?

2.1 · TODO 2 e 3: orders pubblica

TODO 2 · src/events.ts

1. in **setup**: assertExchange, topic, durable
2. in **orderCreated**: ch.publish con persistent, mandatory, messageId
3. aspetta la **callback** in una Promise

TODO 3 · src/routes.ts

Dopo **store.create**:

```
await events.orderCreated(
  order, request.id);
```

in un try/catch: se fallisce, log di errore e **201 lo stesso**.

```
cd services/orders && npm test # all'inizio: # fail 2
```

2.2 · Il primo evento è perso

```
docker compose up --build -d orders
# un ordine da http/shop.http
docker compose logs orders | grep -i evento
# "nessuna coda per questo evento:
# è andato perso"
# "evento OrderCreated pubblicato"
```

RabbitMQ l'ha **preso** (la conferma è arrivata) e l'ha **buttato**: nessuna coda è collegata a order.created.

Nella UI: Exchanges › shop.events › Message rates.

3.1 · TODO 4 e 5: notifications consuma

TODO 4 · src/broker.ts

1. **assertQueue**, durable
2. **bindQueue** a shop.events, order.created
3. **prefetch**
4. **consume** con noAck: false

TODO 5 · src/consume.ts

Caso	Risposta
illeggibile	nack, scarta
handler ok	ack
handler fallisce	nack, requeue

```
cd services/notifications && npm test # all'inizio: # fail 6
```

3.2 · La prima email

```
docker compose up --build -d notifications
# un ordine da http/shop.http, poi:
GET http://localhost:8080/api/v1/notifications
# "subject": "Conferma ordine o-1",
# "body": "Grazie per il tuo ordine o-1.
# - 1 × Tastiera meccanica: 89,90 €
# ...
# Totale: 169,70 €"
```

Nella UI: Queues › **notifications.order-created** › Bindings.

Segui il correlation ID: **docker compose logs | grep** l'x-request-id della risposta. Compare anche in notifications?

4.1 · La coda aspetta

```
docker compose stop notifications
./scripts/ordini.sh 3
# ordine 1: 201 in 0.015 s ...
# UI: notifications.order-created Ready 3
docker compose start notifications
# UI: Ready 0, tre email in /notifications
```

Gli ordini rispondono **in pochi millisecondi** anche con notifications spento.

È la consistenza eventuale: l'ordine c'è subito, l'email prima o poi.

4.2 · Un server di posta inaffidabile

```
MAIL_FAIL_PERCENT=50 docker compose up -d notifications
./scripts/ordini.sh 5
curl localhost:8080/stats/notifications
# {"received":9,"emailsSent":5,"failed":4,
# "redelivered":4,"ignored":0}
```

Ogni fallimento: **nack con requeue**, il messaggio torna in coda e arriva di nuovo, con **redelivered: true**.

Alla fine le email partono tutte. E se il server di posta fosse giù per sempre?

4.3 · Due copie, una coda

```
MAIL_DELAY_MS=2000 PREFETCH=1 \
  docker compose up -d --scale notifications=2
./scripts/ordini.sh 6
docker compose logs -f notifications
# notifications-1 | email inviata cliente1@...
# notifications-2 | email inviata cliente2@...
```

Nella UI, mentre lavorano: **Unacked 2** (uno per copia), gli altri **Ready**.

Ogni email parte una volta sola: le copie si dividono il lavoro.

4.4 · Una caduta a metà lavoro

```
MAIL_DELAY_MS=10000 \
  docker compose up -d --scale notifications=1
# un ordine, poi entro 10 secondi:
docker compose kill notifications
# UI: Unacked 1 → Ready 1
docker compose start notifications
# /stats/notifications: "redelivered": 1
```

Senza ack il messaggio non è perso: alla disconnessione torna **Ready**.

Domanda

E se notifications cadesse **dopo** l'email ma **prima** dell'ack?

Checkpoint

Verifica	Esito atteso
`npm test in orders e notifications`	tutto verde, alcuni saltati
`un ordine`	201 e un'email in /api/v1/notifications
`notifications fermo, 3 ordini`	Ready 3, poi consumati alla ripartenza
`MAIL_FAIL_PERCENT=50`	email tutte inviate, redelivered > 0
`due copie, PREFETCH=1`	le email si dividono tra le copie
`kill a metà lavoro`	il messaggio torna Ready, poi redelivered 1

Consegna: uno screenshot della coda nella UI con messaggi Ready, e l'output di /stats/notifications.

Se qualcosa non va

Sintomo	Causa e rimedio
`depends on undefined service rabbitmq`	depends_on scommentati prima del TODO 1
`UI su 15672 non risponde`	RabbitMQ parte in 10–20 s: docker compose ps
`nessuna coda per questo evento`	Normale prima del TODO 4
`Messaggi sempre Unacked`	Manca l'ack nel TODO 5
`Messaggi Ready, nessun consumer`	Manca consume nel TODO 4, o notifications fermo
`Lo stesso messaggio gira all'infinito`	Handler che fallisce sempre: nack con requeue
`Code sparite`	docker compose down -v cancella il volume

ULTIMI 10 MINUTI

Cosa si rompe se...?

...l'ordine è salvato e orders cade un attimo prima di pubblicare OrderCreated?

...un OrderCreated ha un campo sbagliato, e notifications lancia un errore ogni volta?

...dopo l'email il cliente deve anche pagare, e il pagamento fallisce?

PROSSIMA LEZIONE, L'ULTIMA

Lezione 8 · Pattern avanzati e progetto finale

Retry e Dead Letter Queue, consumer idempotente, Outbox, Saga. Poi tutto insieme: gateway, cache ed eventi. Gli ultimi 30 minuti sono per le demo dei gruppi.

Per la volta prossima: npm test verde e i quattro esperimenti fatti.

PER CHI FINISCE PRIMA, O PER CASA

Esercizi extra

#	Esercizio	Livello	Dove
1	Un ascoltatore nuovo, senza toccare orders	base	Management UI
2	ProductChanged su RabbitMQ	medio	catalog, orders
3	OrderCreated versione 2	medio	notifications
4	Riprovare la pubblicazione	avanzato	orders
5	Comandi o eventi?	carta e penna	—

Testi in lezione-7/extra/README.md. Per 3 e 4 ci sono test pronti; le soluzioni sono in lezione-7/extra/soluzione.

Un ascoltatore nuovo, senza toccare orders

Il reparto vendite vuole vedere tutti gli eventi degli ordini. Solo la Management UI:

1. coda **vendite.tutto**, durable
2. binding da **shop.events** con **order.***
3. due ordini, poi **Get messages**
4. prova anche **#**

Domande

orders è cambiato? E notifications? Cosa succede alla coda se nessuno la legge mai?

ProductChanged su RabbitMQ

Gli avvisi della lezione 6 su **shop.events**, routing key **product.changed**.

- catalog: **rabbit-notifier.ts**, dipendenza amqplib
- orders: coda sua **orders.product-changed**
- compose: RABBITMQ_URL anche per catalog

Domanda

La cache di orders è in memoria: dopo un riavvio è vuota comunque. Quando la coda fa davvero la differenza?

OrderCreated versione 2

Il cliente diventa un oggetto, arriva la valuta: chi legge la v1 si rompe.

```
"version": 2,
"data": { "customer": { "email": "...",
                        "name": "Bruno" },
          "currency": "EUR", ... }
```

notifications accetta v1 **e** v2; una v3 si ignora.

Domande

Si rilascia prima orders o notifications? Per quanto orders deve pubblicare la v1?

Riprovare la pubblicazione

Con RabbitMQ giù oggi l'evento si perde. Un buffer in memoria che riprova ogni 5 secondi:

- **withRetryBuffer(events)** in retry-buffer.ts
- gli eventi in attesa partono **prima** dei nuovi
- oltre 1000: il più vecchio si perde, con un log

Domanda

Cosa succede agli eventi nel buffer se orders si riavvia mentre RabbitMQ è giù?

Comandi o eventi?

Per ognuno: comando o evento, chi lo manda, a chi, cosa succede se arriva due volte.

Messaggio	?
SendEmail	
OrderCreated	
ReserveStock	
StockReserved	
ChargeCard	
PaymentFailed	
ProductChanged	

Spunto

Quali di questi, ripetuti, fanno danni veri?

Per approfondire

RabbitMQ · ***Tutorials* 1–5**	rabbitmq.com/tutorials: code, lavori, pub/sub, routing, topic
RabbitMQ · ***Consumer Acknowledgements and Publisher Confirms***	ack, nack, requeue e conferme in dettaglio
M. Fowler · ***What do you mean by “Event-Driven”?***	martinfowler.com: quattro significati diversi
G. Hohpe, B. Woolf · ***Enterprise Integration Patterns***	Addison-Wesley, 2003: il catalogo dei pattern di messaggistica
amqplib · documentazione	amqp-node.github.io/amqplib: canali, conferme, recovery
Apache Kafka · ***Introduction***	kafka.apache.org: il modello a log, per confronto